Implementation and Study of Evolutionary Algorithms Compared to Non-evolutionary Methods

March 30, 2026

Student: Jair Mendoza
Id: 269615
Instructors: Marcin Komarnicki, Konrad Gmyrek
Class: Optimization Methods: Theory and Applications

Content

Introduction **2**
PFSP 3
Random search 3
Greedy algorithm 3
Simulated annealing 4
Genetic algorithm (GA) 4
Implementation **5**
PFSP 5
Random search 6
Greedy algorithm 6
Simulated annealing 7
Genetic algorithm 7
Results 9
Mutation probability 10
Crossover probability 11
Population size 11
Tournament size 12
Conclusions **12**
Comparisons 12
Tai20_5_0 13
Tai20_10_0 13
Tai20_20_0 14
Tai100_10_0 14
Tai100_20_0 14
Tai500_20_0 14

Introduction

The following descriptions of the topics are done in a simple and general way since the objective of this report is to show the implementation of them.

PFSP

The Permutation Flowshop Scheduling Problem (PFSP) is an NP-hard problem, the core idea of the problem is that we have J jobs that should be performed in M machines. Each job has a specific computational time for each machine. The goal is to find a job sequence that gives us the lowest possible time needed for the jobs to be performed in all machines. The main constraint is that for a j job to be processed on a m machine, the previous j job must be completed by the previous m machine first.

This report handle cases where the values are always for $j > 1$ and $m > 1$ thus the formula for the completion time is as follows:

$$c(xj,\ m) = max\{c(xj - 1,\ m), c(xj,\ m - 1)\} + p(xj,\ m) \qquad (1)$$

where c(xj , m) is the completion time of the xj th job on the mth machine and p(xj , m) denotes the processing time of job xj on machine m.

This report used the dataset provided by Taillard, specifically:

- tai20_5_0, tai20_10_0, and tai20_20_0.
- tai100_10_0 and tai100 20_0.
- tai500_20_0.

Random search

As its name suggests, this algorithm iterates over randomly selected solutions from the solution space (all possible sequences of jobs) and keeps the best one. This is the simplest algorithm used in the report.

Greedy algorithm

Greedy algorithm is basically a random search algorithm but implemented on each node of the solution rather than to the whole of it. Let a solution be a sequence of nodes $S = (n_1, n_2, n_3, ..., n_k)$, if the algorithm starts at the node n_1 the next node would be described as:

$$n_{i+1} = \min(w(n_{i,}v))$$

where $w(n_{i,}v)$ is the weight and v is the set of remaining nodes. Due to its nature, this algorithm is deterministic.

Simulated annealing

This algorithm works very similar to the hill climb algorithm, it is basically the same but with an additional parameter for accepting worse solutions. It starts by choosing a random solution from the solution space, then it proceeds to check neighbours solutions, if one is better it immediately becomes the next solution but if it is worse then it may be accepted if the probability allows it. The probability is given by formula:

$$p = e^{\Delta E/T} \tag{2}$$

where ΔE is the difference between solution values and T is the heat parameter. Each iteration the probability T decreases based on the parameter which is usually called temperature.

If we visually imagine the algorithm giving jumps between the solution space, then the higher the T is the bigger the jumps it will take as the next step and as T is decreasing (cooling down), the jumps become smaller. This behaviour is great to avoid local minimums which is the exact problem the Hill climbing algorithm has.

Genetic algorithm (GA)

This is the main focus of this report. The genetic algorithm GA works by choosing and adjusting few parameters:

Population**:** Selected solutions that will act as parents.

Generations: Number of generations we want to have.

Selection parameters: The main ways for parent selection are tournament and roulette.

Mutation probability: How frequent the mutation is going to happen.

The usual flow of the algorithm is: First, the population is initialized. Parent candidates are then chosen; for example, in Tournament Selection, a fixed number of solutions are randomly picked from the population, and the best among them is selected. This is repeated until the desired number of parents is reached.

Secondly we generate the childs. Two parents are chosen based on the crossover operator. There are two types of crossover: OX (Ordered Crossover) which is the easiest to implement and PMX (Partially Matched Crossover) which is a bit more complex. OX works by taking a random sequence from Parent 1 and copies it to the same position in the child. The remaining spots are filled using the sequence from Parent 2. This process respects specific problem constraints, such as preventing duplicate values in the sequence.

Lastly we have mutation, this can be of two forms Swap and Inversion. For swap, two genes in a chromosome are selected and its values are swapped. In inversion, two random positions in the sequence are selected, and the order of the chromosomes within this selected segment is reversed.

Implementation

The implementation was primarily done in Python and later was moved to C++. Python was used to quickly sketch the algorithms and C++ to save time on the tai500_20_0.fsp dataset.

PFSP

The presented function code implements the formula (1):

```python
subject = [Job(id=i, times=t) for i, t in enumerate(subject)]
def solution(jobs, machines):
    time = [0] * machines
    for jb in jobs:
        time[0] += jb.times[0]
        for i in range(machines):
            time[i] = max(time[i], time[i - 1]) + jb.times[i]
    return time
sol = solution(subject, machines)
```

As it is visible the subject is an indexed list of jobs ids and its processing times. This was a bit confusing to work with, that is why in the c++ version it is handled quite differently. In c++we are working with a class named Subject_solution that have a method to calculate the makespan:

```cpp
void compute_makespan() {
    int num_machines = (int)this->permutation[0].times.size();
    std::vector<int> time(num_machines, 0);
    for (const auto& job : this->permutation) {
        time[0] += job.times[0];
        for (int i = 1; i < num_machines; i++) {
            time[i] = std::max(time[i], time[i - 1]) + job.times[i];
        }
    }
   this->makespan = time[num_machines - 1];
}
```

In the c++ version, the class Subject_solution handles the indexing, jobs, times, machines, making the algorithm's syntax readability much more efficient (the python version also implements a class but it was much poorly handled). The python version returns a list of makespan of each job while the C++ computes only the total makespan .

Random search

This implementation is pretty much straightforward, it starts the loop and shuffles the given sequence and computes the makespan. This makespan is compared with the Highest_solution found and updates it if it finds a new one. It will return the best sequence of jobs and makespan.

```python
def randomSearch(subject, machines):
    Highest_solution = float("inf")
    Best_subject = None
    for i in range(10000):
        rand.shuffle(subject)
        sol = solution(subject, machines)
        if Highest_solution > sol[-1]:
            Highest_solution = sol[-1]
            Best_subject = subject.copy()
    return Best_subject, Highest_solution
```

Greedy algorithm

Here we remove the first element of the job sequences and pass the removed element to another list named *Best_subject*. The algorithm then starts to compare which elements of the remaining solutions together with the *Best_subject* gives the best makespan and appends the element which has the best makespan and repeats this until there are no more remaining elements.

```python
def greedy(subject, machines):
    temp_subject = subject.copy()
    Best_subject = [temp_subject.pop(0)]
    rand.shuffle(temp_subject)
    while temp_subject:
        current_sol = float("inf")
        current_job = None
        for s in temp_subject:
            current_step = Best_subject + [s]
            ms = solution(current_step, machines)[-1]
            if ms < current_sol:
                current_sol = ms
                current_job = s
        Best_subject.append(current_job)
        temp_subject.remove(current_job)
    return Best_subject, current_sol
```

Simulated annealing

There is a function made for the genetic algorithm for swapping a given solution which is being used here to reproduce neighbours. The code sets an initial solution and iterates over its neighbours. The condition for accepting a worst solution is by generating a random number between 0-1 and comparing it with the probability given by the formula (2) (if the probability is 0.9 then there is 90% chance for the generated number to be in this range).

```python
def simulatedAnnealing(
    subject, machines, temp=500, cooling_rate=0.999, iterations=10000):
    rand.shuffle(subject)
    current_sol = solution(subject, machines)[-1]
    current_subject = Subject_solution(subject, current_sol)
    for _ in range(iterations):
        temp_subject = current_subject.copy()
        swap(temp_subject, machines)
        if temp_subject.makespan < current_sol:
            current_sol = temp_subject.makespan
            current_subject = temp_subject.copy()
        elif rand.random() < 2.71828 ** ((current_sol-temp_subject.makespan)/temp):
            current_sol = temp_subject.makespan
            current_subject = temp_subject.copy()
        temp *= cooling_rate
    return current_subject
```

After each iteration the temperature value is decreased and updated. For better handling of the data in this and the next algorithm the class Subject_solution that manages the solution, makespan and ids of jobs is used.

Genetic algorithm

The method used to generate childs is the ox ordered crossover, the parents P1&P2 are objects of the Subject_solution class. Ox takes the indexes of the job sequence in the solution of parent P1 and proceeds to randomly take a portion from it and paste it into the child, then it iterates over the indexes of the parent P2 and copies them into the child unless the job is already there.

The mutation is of swap type, the logic is simple we just take two jobs from the solution, swap their values and calculate the new makespan.

These two functions are methods in the C++ class version of the code. This in order to maintain more clarity among the code.

Below are the functions and class code in python:

```python
class Subject_solution:
    job_lookup = {}
    def __init__(self, permutation, makespan):
        self.permutation = permutation
        self.makespan = makespan
    def copy(self):
        return Subject_solution(self.permutation.copy(), self.makespan)
    @classmethod
    def set_lookup(cls, subject):
        cls.job_lookup = {job.id: job for job in subject}
    @classmethod
    def from_ids(cls, child_ids, machines):
        perm = [cls.job_lookup[id] for id in child_ids]
        ms = solution(perm, machines)[-1]
        return cls(perm, ms)
    @property
    def ids(self):
        return [job.id for job in self.permutation]
```

from_ids() function is used to generate an object based on the index sequence.

```python
def ox(parent1, parent2, machines):
    child_ids = [None for _ in range(len(parent1))]
    i = rand.randint(0, len(parent1) - 1), j = rand.randint(0, len(parent1) - 1)
    if i > j:
        i, j = j, i
    child_ids[i:j] = parent1[i:j]
    for gene in parent2:
        if gene not in child_ids:
            for k in range(len(child_ids)):
                if child_ids[k] is None:
                    child_ids[k] = gene
                    break
    return Subject_solution.from_ids(child_ids, machines)
```

```python
def swap(subject, machines):
    i = rand.randint(0, len(subject.ids) - 1)
    j = rand.randint(0, len(subject.ids) - 1)
    if i > j:
        i, j = j, i
    subject.permutation[i], subject.permutation[j] = (
        subject.permutation[j],
        subject.permutation[i],)
    subject.makespan = solution(subject.permutation, machines)[-1]
```

Once these functions are implemented the GA flow becomes much more easy to understand. First the code initializes the population which is just a list of Subject_solution objects, then the tournament selection is started which overwrites the parents into the population list. Secondly, we take the parents and start producing the child by taking parents in pairs. Finally, we iterate through all the children and mutate them according to the mutation probability.

```python
def genetic(subject,machines,population_size=100,generations=100,
    root_parents=20,tournament_size=10,pm=0.1,):
    Population = []
    population_makespan = []
    temp_subject = subject.copy()
    for i in range(population_size):
        ... # generates population
    for gen in range(generations):
        Parents = []
        Children = []
        # Tournament selection
        for _ in range(root_parents):
            tournament = rand.sample(Population, tournament_size)
            best_makespan = float("inf")
            best_parent = None
            for individual in tournament:
                if individual.makespan < best_makespan:
                    best_makespan = individual.makespan
                    best_parent = individual
            Parents.append(best_parent)
        # Crossover ox
        for i in range(0, len(Parents), 2):
            if i + 1 < len(Parents):
                Children.append(ox(Parents[i].ids, Parents[i + 1].ids, machines))
                Children.append(ox(Parents[i + 1].ids, Parents[i].ids, machines))
        # Mutation swap
        for child in Children:
            if rand.random() < pm:
                swap(child, machines)
        Population = Children.copy()
    for s in Population:
        ... # picks the best subject from the population
    return Population, best
```

Results

The datasets used in the report were:

- Basic: *tai20_5_0, tai20_10_0, and tai20_20_0.*
- Medium: *tai100_10_0 and tai100_20_0.*
- Hard: *tai500_20_0.*

The code is designed to run and test 20 instances of 10000 iterations for each algorithm, the time difference between python and c++ was not very noticeable in small data (Basic and Medium) but it was clearly different in the Hard dataset since it was the longest. Below is a comparison between the completion times it takes on average.

	Tai20_20_0.fsp [s]	Tai100_20_0.fsp [s]	Tai500_20_0.fsp [s]
C++	~10	~50	~272
Python	~12	~63	~824

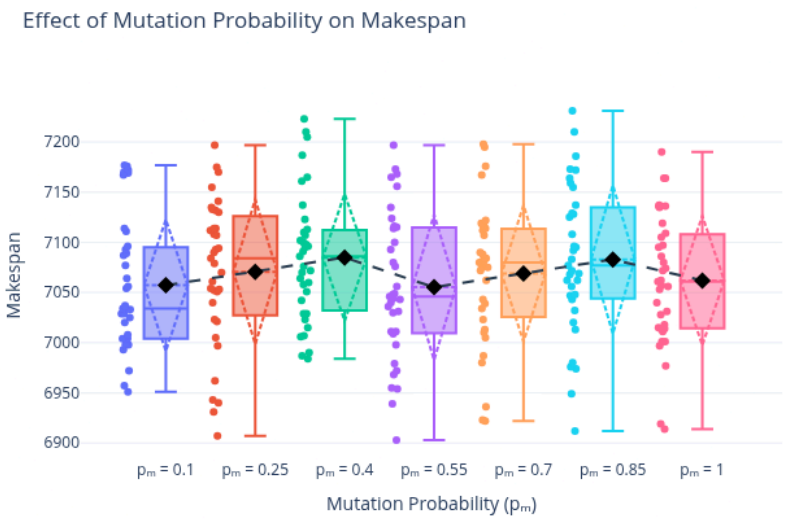
For the GA algorithms the parameters that can be tuned and its default values are the following:

- pop size (*population size*) = 100.
- gen (*number of generations*) = 100.
- Px (*crossover probability*) = 0.7.
- Pm (*mutation probability*) = 0.1.
- Tour (*tournament size*) = 10.

This parameter has been tuned and its influence has been collected with 35 instances of the genetic algorithm.

Mutation probability

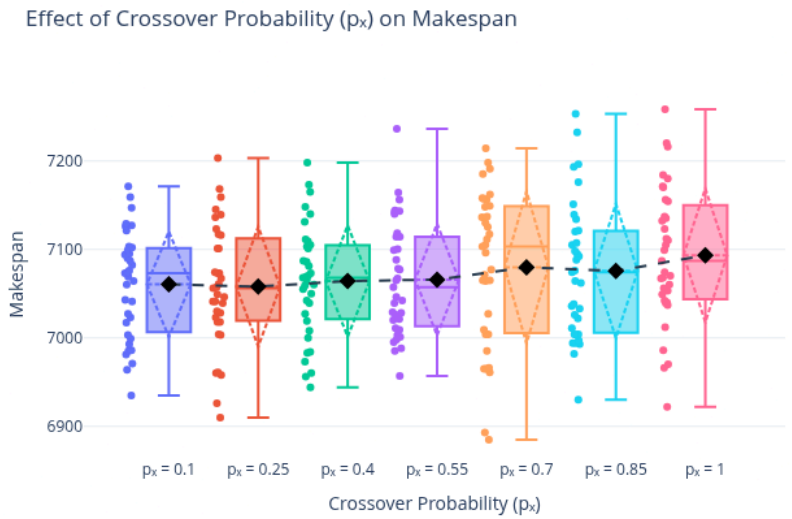
Below is a graph that shows the influence of the *pm* on the graph.



From this graph we can see that the best *pm* is below 0.25 since the solutions in that range are very consistent and yet have a good markspan. Values outside this range may have the chance to give better results but they have a very high probability to give a very bad one as well. It is very interesting to note that *pm* = 0.4 and 0.85 have the highest makespan values.

Crossover probability

Taking *pm=1* and *tournament size=10*, the influence of crossover probability *px* look like



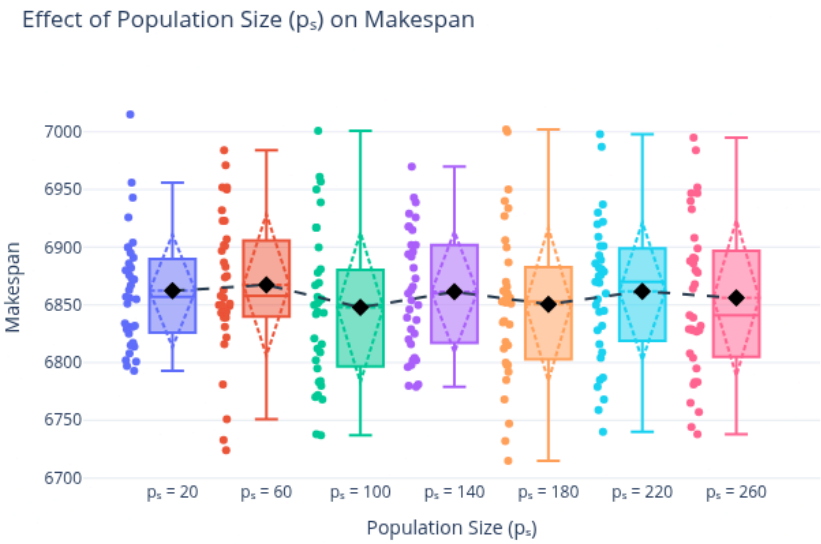
follows:

From the chart it is clear that the safest *pm* is 0.25 since it has a reasonable low makespan while keeping a low ceiling.

It is worth noting that with *pm=0.7* we get the best possible result but there is a high probability that many bad solutions may be generated however those bad markspans are not as high as other parameters . The following charts are going to take this parameter into consideration.

Population size

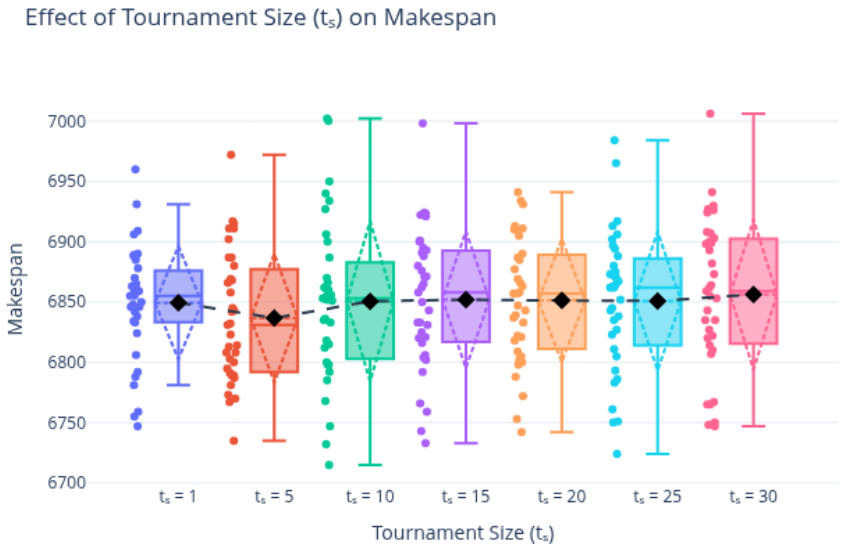
The following charts takes *pm=1, px=0.25* and, *tournament size=10*:



The best range is: $100 \leq ps \leq 180$. This report takes *ps*=180 since it may give the best result.

Tournament size

The following charts takes *pm=1, px=0.25* and, *population size=180*:



Here the clearly best value is $5 \leq pm \leq 10$, being *pm*=5 the most reliable.

Conclusions

The GA may be incorrectly implemented since the changes in the parameters although they differ from each other they resemble more like a random distribution rather than one that is directly affected by the parameters. Secondly, the SA algorithm constantly beats the GA so this may be another indicator that the algorithm is incorrectly implemented.

Comparisons

The parameters used for the Genetic Algorithm are:

- pop size (*population size*) = 180.
- gen (*number of generations*) = 100
- Px (*crossover probability*) = 0.25
- Pm (*mutation probability*) = 0.1
- Tour (*tournament size*) = 5

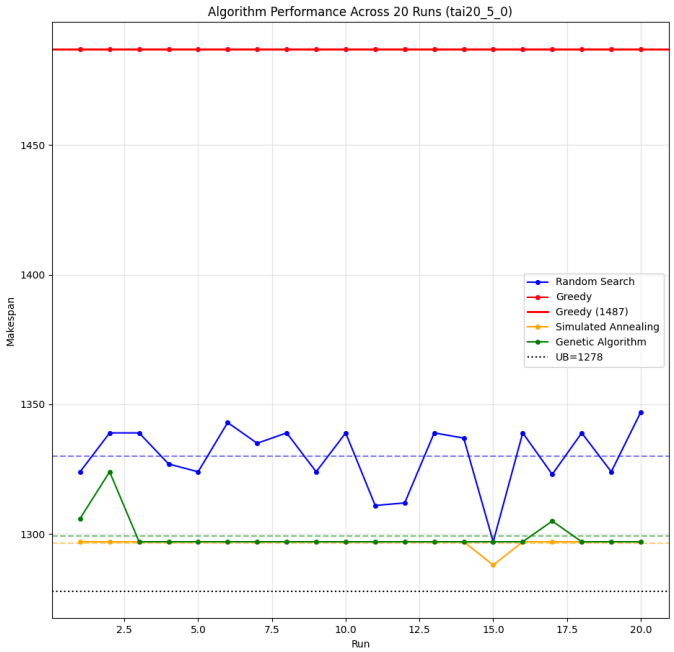
For the Simulated Anneling are:

- T (*Temp*) = 500.0.
- *cooling_rate* = 0.999.

The following charts shows the comparison between the performance of all the algorithms in the datasets: tai20_5_0, tai20_10_0, tai20_20_0, tai100_10_0, tai100_20_0 and tai500_20_0.

Tai20_5_0

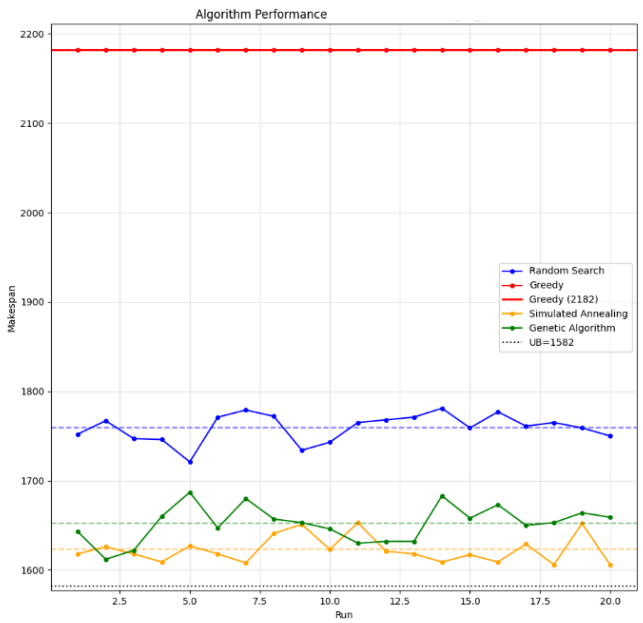
The greedy algorithm is always the same for the Basics datasets. This will change in other dataset to see how it behaves with a randomized initialization.



Here it is visible how the two metaheuristic algorithms consistently find a very good solution. It is worth noting that in this run only SA was able to positively escape the local minima while the GA does escape local minima but to a worse solution.

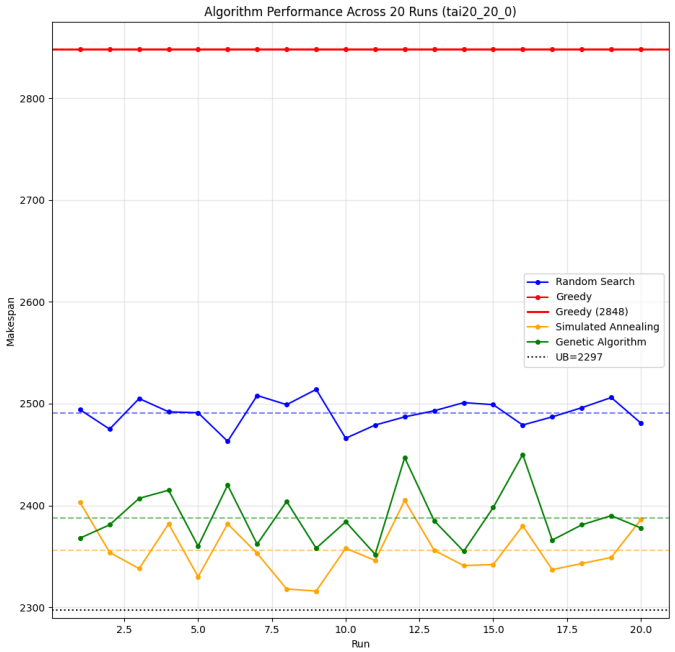
Tai20_10_0

In general the SA is a bit more constant than the GA.



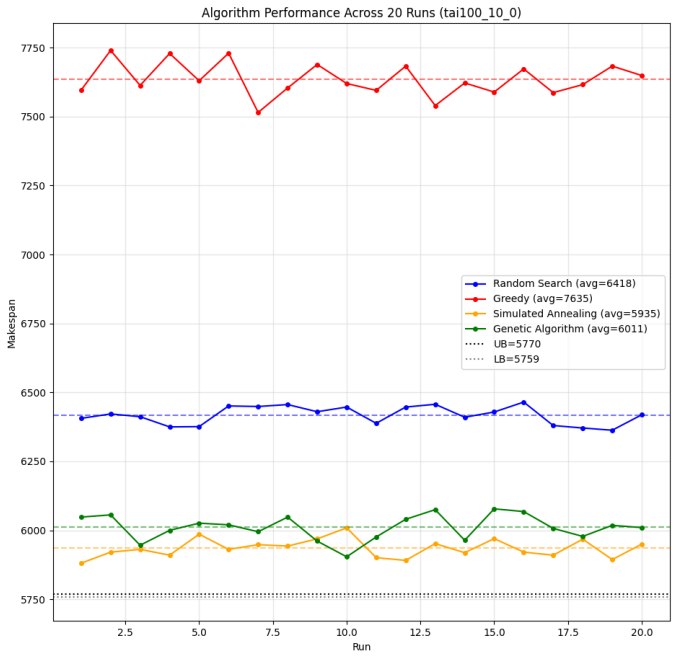
Tai20_20_0

It is interesting to note that the best solutions achieved by the GA are the same that the average solution SA would give. SA is the best performing solution so far.

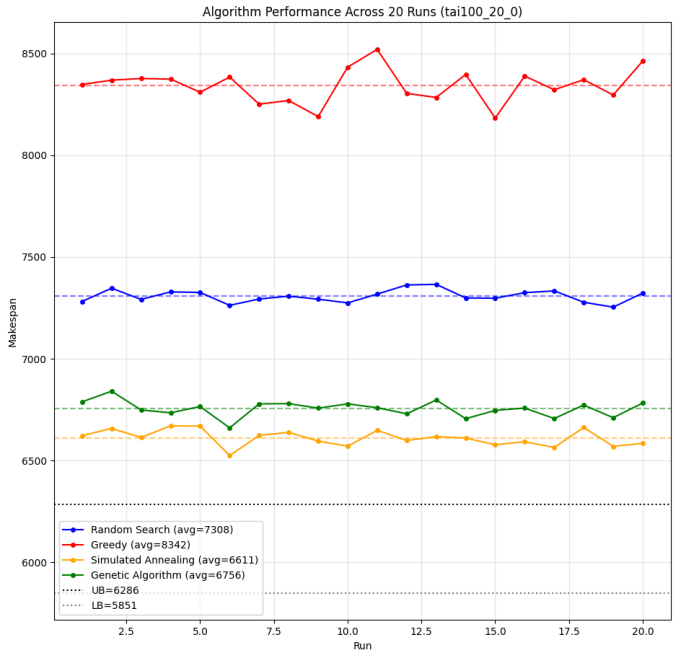


Tai100_10_0

Here and in the following charts, the greedy algorithm is randomly initialized, it still is deterministic but the solutions now can vary depending on the initial condition.

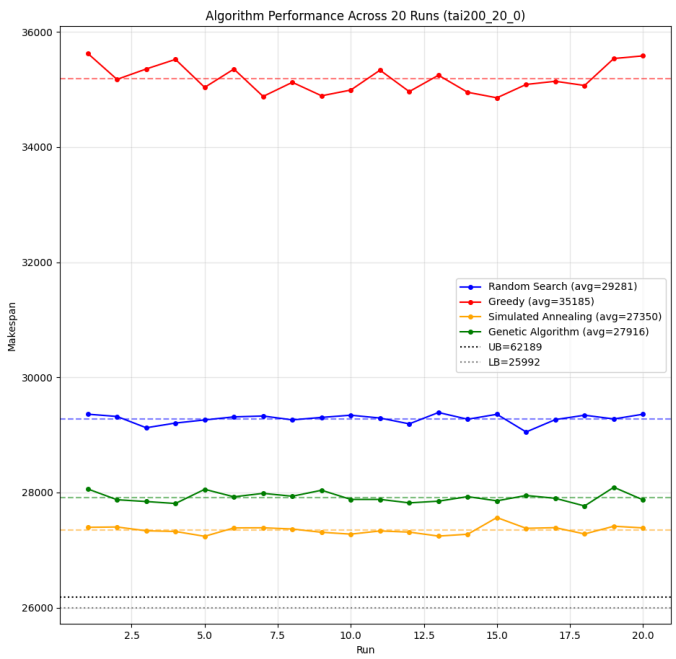


Tai100_20_0



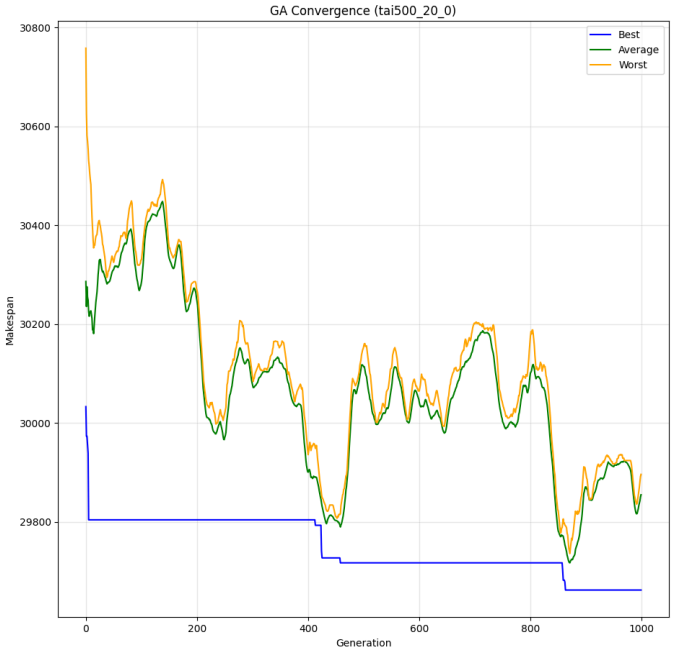
Tai500_20_0

Here it is worth to note that GA and SA don't cross each other anymore, they are close but the worst value by the SA is better than the best value by the GA:



Convergence graph

Below is a GA convergence on tai100_20_0 (100 jobs, 20 machines):



Configuration: population=180, generations=1000, tournament=5, Pm=0.1, Px=0.8, OX crossover, swap mutation, 10% elitism

The figure shows the evolution of the best, average and worst makespan values across 1000 generations of the Genetic Algorithm.

The best solution (blue) follows a characteristic staircase pattern — remaining flat for many generations then dropping sharply when a new best permutation is discovered. This reflects the GA's ability to occasionally escape local optima through crossover and mutation. The most significant improvements occur before generation 600, after which the best solution stabilizes around 7340, suggesting the algorithm has converged to a strong local optimum.

The average (green) and worst (orange) lines follow a general downward trend with visible oscillation throughout — this is expected and healthy behavior, indicating the population maintains diversity rather than collapsing to a single solution. The gap between best and average remains substantial, confirming that elitism successfully preserves top solutions while the rest of the population continues exploring.

Notable observation — after generation 600 the best line flattens completely while average and worst continue fluctuating, suggesting that further generations yield diminishing returns for this instance and configuration. This points to 600 generations as a reasonable stopping criterion for this problem size.